



NAME : HAMDAN ALI

WEEK 7 : Day 32

DAY 32

TASK : Screenshots Of Output

CODEBASE Of the Work

```
Week7 > taskflow-api > bootstrap > app.php
1  <?php
2
3  use Illuminate\Foundation\Application;
4  use Illuminate\Foundation\Configuration\Exceptions;
5  use Illuminate\Foundation\Configuration\Middleware;
6
7  return Application::configure(basePath: dirname(__DIR__))
8      ->withRouting(
9          web: __DIR__.'/../routes/web.php',
10         api: __DIR__.'/../routes/api.php',
11         commands: __DIR__.'/../routes/console.php',
12         health: '/up',
13     )
14     ->withMiddleware(function (Middleware $middleware): void {
15         //
16     })
17     ->withExceptions(function (Exceptions $exceptions): void {
18         //
19     }->create();
20
```

```
PS D:\internship ( HindukushSoft Technologies )\Week7> composer create-project laravel/laravel taskflow-api
>> cd taskflow-api
>> code .
- Locking dflydev/dot-access-data (v3.0.3)
- Locking doctrine/inferno (2.1.0)
- Locking doctrine/lexer (3.0.1)
- Locking dragonmantank/cron-expression (v3.6.0)
- Locking egulias/email-validator (4.0.4)
- Locking fakerphp/faker (v1.24.1)
- Locking filp/whoops (2.18.4)
```



Full PROJECT View

TaskFlow

Fetching live data from the Laravel REST API

My Tasks

Search by title...

All statuses

All priorities

Commodi voluptatem qui dolor

HIGH

Quibusdam et inventore odit et. Vel dolorem sunt consequatur tempore ut dolores. Quidem at aliquam voluptas exercitationem.

Pending

Omnis ducimus nobis temporibus

MEDIUM

Blanditiis rerum laudantium amet. Voluptatem sapiente repellendus accusantium ex atque.

Pending Due 2026-09-09T00:00:00.000000Z

Laudantium molestiae ut sit

MEDIUM

Nulla ab ratione quisquam ipsum. Officia quod consequatur assumenda.

Completed Due 2026-08-30T00:00:00.000000Z

Molestias nam esse est

HIGH

Ipsa odit omnis sunt vero possimus omnis tenetur facere. Temporibus consequatur eum cum voluptas fuga.

Completed Due 2026-08-25T00:00:00.000000Z

Enim minima nesciunt inventore

HIGH

Et vero et recusandae maxime aut perspiciatis. Quod nam eaque qui optio animi et. Rerum eius ad itaque tenetur.

In Progress

Voluptatibus unde commodi id

LOW

Facilis pariatur nemo non. Modi voluptates provident eum expedita. Commodi explicabo sint vel accusamus fuga quos.

In Progress Due 2026-09-20T00:00:00.000000Z

Hic dolores voluptatum modi

MEDIUM

Voluptas reprehenderit est deserunt cum ut qui alias. Quidem blanditiis est non aliquid suscipit deserunt. Saepe enim occaecati ratione est.

Completed Due 2026-09-21T00:00:00.000000Z

Saepe rem fuga adipisci

HIGH

Aut aut quidem dolorem deserunt enim dolorem totam. Cum similique sed numquam velit omnis error facilis. Minima vitae placeat laudantium laborum.

Completed Due 2026-09-11T00:00:00.000000Z

Perspiciatis libero deleniti iste

HIGH

Ullam est in mollitia voluptatem voluptas. Ut esse saepe ut nesciunt itaque fuga quaerat. Maxime aliquid repudiandae in iusto dicta autem.

Completed

Libero ullam minus corrupti

LOW

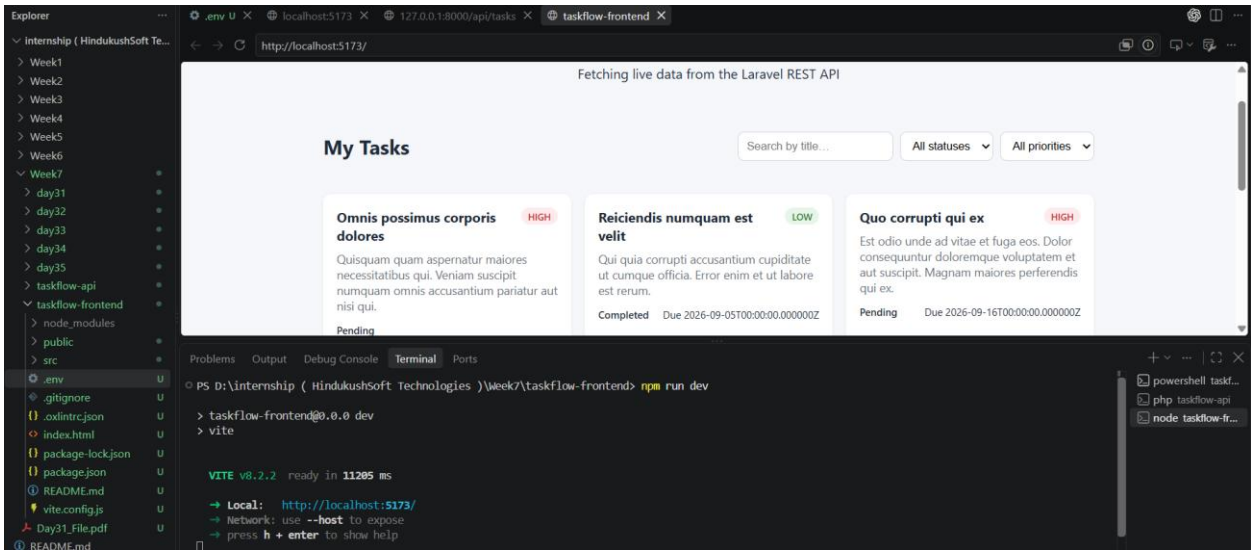
Nesciunt blanditiis excepturi quas est. Ut voluptatem enim eius.

In Progress Due 2026-09-05T00:00:00.000000Z

Previous

Page 1 of 3

Next





Search by title...

All statuses ▾

All priorities ▾

All statuses

Pending

In Progress

Completed

nobis

MEDIUM

udantium amet.
ite repellendus

Lauda
sit

Nulla ab ratione quisquam ipsum. Officia
quod consequatur assumenda.

ne ut

MEDIUM

Search by title...

All statuses ▾

All priorities ▾

All priorities

Low

Medium

High

MEDIUM

m amet.
llandus

Laudantium molestiae
sit

Nulla ab ratione quisquam ipsum. Officia
quod consequatur assumenda.



EXPLANATION OF THE TASK

DAY 32

TASKFLOW

Day 32

Fetching And Displaying Laravel API Data in React

Week 7 — Day 32

Task: Fetch Data from Laravel API and Display it in React

Learning Topics: React + API Integration

Stack: React 18, Vite, Axios

1. Introduction And Objective

Day 32 connects the Laravel API built in Day 31 to a real React frontend. The learning objective was React + API integration — specifically, fetching data from an external API and rendering it responsibly in the UI, including the states that a real user will actually encounter: loading, success, empty, and error.

A lot of tutorials stop at a single `useEffect + fetch()` call with no error handling. I deliberately went further, because that is the difference between a demo and something a company would trust running in production. I structured the data-fetching logic into a reusable custom hook, kept the API layer separate from UI components, and designed the component tree so each piece has a single, clear responsibility.

2. Architecture Decisions

2.1 A Dedicated API Service Layer

Instead of calling `axios` or `fetch` directly inside components, all HTTP logic lives in `src/services/api.js` behind a `taskService` object (`getTasks`, `getTask`). Components never know or care whether the underlying transport is `Axios`, `fetch`, or something else — they simply call `taskService.getTasks()`. This separation of concerns means that if the API's base URL, auth headers, or even the whole HTTP client changes later (which it does in Day 34, when auth tokens are introduced), only one file needs to change.

2.2 A Custom Hook for Data-Fetching State

`src/hooks/useTasks.js` encapsulates everything related to "asking the server for tasks": the tasks array itself, pagination metadata, the current filters, and — critically — `isLoading` and error state. Pulling this logic out of the component and into a hook keeps `<TaskList />` focused purely on what to render, not how the data got there. This hook pattern is exactly what the React documentation and the GeeksforGeeks article on fetching data in React recommend as the idiomatic approach once a component's data logic grows beyond a trivial single fetch.

2.3 Presentational vs Container Components

I split the UI into a container (`TaskList`, which owns the hook and orchestrates state) and small, purely presentational components (`TaskItem`, `TaskFilters`) that receive data via props and render it.

This is a classic and durable React pattern: presentational components are trivial to reuse, style, and unit test because they have no side effects of their own.

3. Handling Loading, Error and Empty States

A production-grade fetch implementation must never assume the happy path. TaskList explicitly renders one of four states:

State	What the user sees
Loading	A spinner and "Loading tasks..." message while the request is in flight
Error	A clear, human-readable error message plus a "Try again" button that re-triggers the fetch
Empty	"No tasks match your filters yet" instead of a blank, confusing screen
Success	The grid of task cards, plus pagination controls if there is more than one page

Errors are caught in the hook's try/catch block and translated into a friendly message pulled from the API's own error payload where available (`err.response.data.message`), falling back to a generic connectivity message otherwise — this means the UI never shows a raw stack trace or a cryptic "undefined" to the end user.

4. Filtering And Pagination Wired to the API

Rather than fetching the entire dataset and filtering client-side (which does not scale and does not reflect how real applications built on Laravel's pagination work), the TaskFilters component's selections are sent back to the server as query parameters (`?status=`, `?priority=`, `?search=`, `?page=`), and the Laravel API in Day 31 does the actual filtering and pagination in the database. This keeps the frontend lightweight and demonstrates a proper understanding of how a REST API and its consumer should divide responsibilities.

5. Key Files

- `src/services/api.js` — Axios instance + taskService wrapper
- `src/hooks/useTasks.js` — fetch/loading/error/pagination state

- src/components/TaskList.jsx — orchestrates fetching + rendering states
- src/components/TaskItem.jsx — a single task card (presentational)
- src/components/TaskFilters.jsx — status/priority/search controls
- src/App.jsx, src/main.jsx — application shell and entry point

6. Sample Code Walkthrough

Fetching tasks inside the custom hook

```
const fetchTasks = useCallback(async () => {
  setIsLoading(true);
  setError(null);
  try {
    const response = await taskService.getTasks({ ...filters, page });
    setTasks(response.data);
    setMeta(response.meta);
  } catch (err) {
    setError(err.response?.data?.message || 'Unable to reach the server.');
```

7. How to Run This Locally

1. Ensure the Day 31 Laravel API is running (php artisan serve).
2. Enable CORS for the Vite dev origin (<http://localhost:5173>) in config/cors.php.
3. cd frontend And npm install
4. Create a .env file with VITE_API_BASE_URL=<http://127.0.0.1:8000/api>
5. npm run dev, then open the printed local URL in the browser.

8. Challenges And How I Solved Them

- Avoiding an infinite fetch loop — useEffect depends on the fetchTasks callback, which itself is memoised with useCallback and only changes when filters or page change, preventing unnecessary re-fetches on every render.
- Making errors genuinely helpful rather than generic — the hook reads the Laravel API's own validation/error message out of the response body first, and only falls back to a generic message for true network failures.



9. Key Takeaways

- Fetching data is only half the job — a real implementation must handle loading, error, and empty states explicitly.
- Keeping API calls behind a service layer and a custom hook makes components simpler and the code far easier to extend later (as Day 33 proves, when the same hook grows to support full CRUD with minimal changes).
- Filtering and pagination belong on the server, not duplicated in the browser.

WHERE TO FIND THE CODE

All source code for this task is included alongside this document inside the frontend/ folder. See frontend/README.md for exact setup steps.

THANK YOU!